

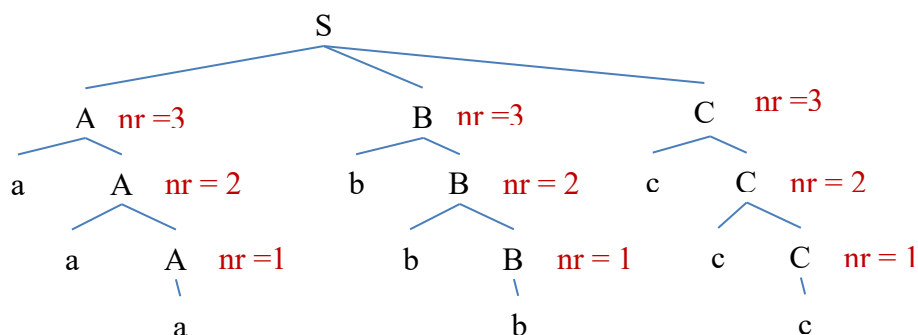
Gramatici de atribute

1. Fie limbajul: $L = \{a^n b^n c^n \mid n - \text{natural}\}$
 Dati o gramatica de atribute care il genereaza.

		Reguli semantice	
$S \rightarrow \epsilon$			
$S \rightarrow ABC$		$A.nr = B.nr$ $A.nr = C.nr$	Predicat - Regula semantica este folosita ca o conditie.
$A_0 \rightarrow a A_1$		$A_0.nr \leftarrow A_1.nr + 1$	Regula de evaluare
$A \rightarrow a$		$A.nr \leftarrow 1$	
$B_0 \rightarrow b B_1$		$B_0.nr \leftarrow B_1.nr + 1$	
$B \rightarrow b$		$B.nr \leftarrow 1$	
$C_0 \rightarrow c C_1$		$C_0.nr \leftarrow C_1.nr + 1$	
$C \rightarrow c$		$C.nr \leftarrow 1$	

Exemplu:

Pentru cuvantul aaabbbccc, arborele de derivare si valorile atributelor arata astfel:



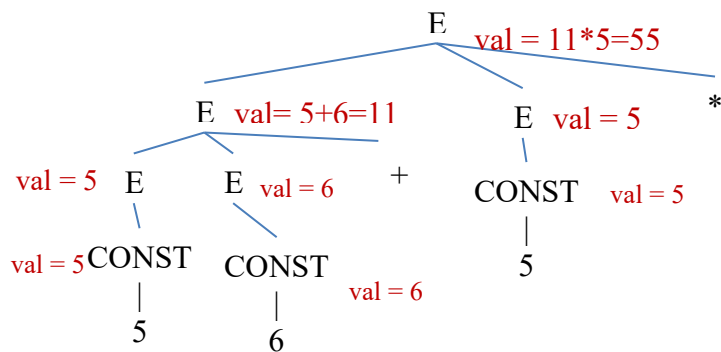
2. Descrieti o gramatica de atribute care determina valorile expresiilor aritmetice in forma postfixata. EX: 5 6 + 5 *

Observatii:

- Vom da GIC care generează expresii aritmetice în forma postfixată și adaugam atributul val.
- Evaluarea atributelor (pe arborele de derivare) se face de jos in sus.
- Vom presupune ca singurele valori posibile sunt 5 si 6

(reguli de prod.		atr.)	
$E \rightarrow E' E'' +$	(1)	$E.val \leftarrow E'.val + E''.val$	
$E \rightarrow E' E'' *$	(2)	$E.val \leftarrow E'.val * E''.val$	
$E \rightarrow CONST$	(3)	$E.val \leftarrow CONST.val$	
$CONST \rightarrow 5$		$CONST.val \leftarrow 5$	2 reguli de productie adaugate pentru ca gramatica sa fie completa. Ele suplinesc (/inlocuiesc) informatiile care, in mod normal, vin de la partea de analiza lexicala.
$CONST \rightarrow 6$		$CONST.val \leftarrow 6$	

EX: $5\ 6 + 5\ *$



3. Descrieti o gramatica de attribute care, pentru o expresie aritmetica data in forma infixata, determina expresia aritmetica in forma postfixata. Exemplificati evaluarea atributelor pentru expresia: $a + b * c$

Rezolvare partiala:

reguli de prod.	Reguli semantice
$E1 \rightarrow E2 + T$	$E1.code \leftarrow E2.code \parallel T.code \parallel '+'$
$E \rightarrow T$	$E.code \leftarrow T.code$
$T1 \rightarrow T2 * F$	$T1.code \leftarrow T2.code \parallel F.code \parallel '*'$
$T \rightarrow F$	$T.code \leftarrow F.code$
$F \rightarrow (E)$	$F.code \leftarrow E.code$
$F \rightarrow id$	$F.code \leftarrow id.varn$
$id \rightarrow a$	$id.varn \leftarrow 'a'$
$id \rightarrow b$	$id.varn \leftarrow 'b'$
$id \rightarrow c$	$id.varn \leftarrow 'c'$

4. Dati o gram. de attribute care defineste semantica limbajului numerelor rationale pozitive, scrise cu "." in baza 2.

5. Fie următoarea gramatică independentă de context

$N \rightarrow B N$

$N \rightarrow B$

$B \rightarrow 0$

$B \rightarrow 1$

Atribuați gramaticii:

atributul v – cu semnificația “valoarea ca număr” întreg fără semn

Arătați cum se evaluează atributul pentru secvența 101

(*Puteti adauga si alte attribute daca este necesar.*)

6.

a) Dați o gramatică care generează secvențe de 0 și 1 în care nr. de 0 este par și nr. de 1 este par.

b) Fie attributele cu semnificația: $nr0$ este numărul de 0 și $nr1$ este numărul de 1 dintr-o secvență dată. Asociați attributele simbolurilor gramaticii și definiți regulile de evaluare.

c) Arătați cum se evaluează atributul pentru secvența 0101

(*Puteti adauga si alte attribute daca este necesar.*)

7. **Numarotarea pozițiilor**

Extinde gramaticile pentru declararea tipului variabilelor astfel încât fiecare identificator din lista să primească un atribut pos (poziția în listă). Tipurile posibile vor fi char, int, double. Numele posibile pentru identificatori vor fi: a, b, c. Numararea începe de la 1.

Arătați cum se evaluează atributul pentru secvența int a, b, c ;

Rezolvare parțială:

(reguli de producție & reguli semantice)

$\langle decl_tip \rangle \rightarrow \langle tip \rangle \langle lista_id \rangle ;$

$\langle tip \rangle \rightarrow int$

$\langle tip \rangle \rightarrow char$

$\langle tip \rangle \rightarrow double$

$\langle lista_id \rangle_0 \rightarrow \langle lista_id \rangle_1 , id$

$id.pos \leftarrow \langle lista_id \rangle_1 + 1$

$\langle lista_id \rangle_0 \leftarrow \langle lista_id \rangle_1 + 1$

$\langle lista_id \rangle \rightarrow id$

$id.pos \leftarrow 1$

$id \rightarrow a$

$a.pos \leftarrow id.pos$

$id \rightarrow b$

$b.pos \leftarrow id.pos$

$id \rightarrow c$

$c.pos \leftarrow id.pos$

8. Dați o gramatică pentru propoziții simple, cu subiect și predicat, în care subiectul este un substantiv articulat, hotărât sau nehotărât. Alfabetul limbajului este: caine, cainele, pisica, pantera, urs, ursul, un, o, alearga, doarme, inclusiv “.” (punctul). Obs.: textul este scris (intentionat) fără diacritice

Atribuați gramaticii astfel încât ea să genereze numai propoziții corecte și să poată fi extinsă ușor cu alte substantive. Folosiți $\leftarrow$ acolo unde regula semantică este o condiție și $\leftarrow$ acolo unde ea este o regulă de evaluare

Exemplu propoziții corecte
(care trebuie generate):

Exemple propoziții incorecte:

Un caine alearga.
Cainele alearga.

Un cainele alearga

Gramatica va include urmatoarele reguli de productie :

Det → un	Subst → caine	Verb → alearga
Det → o	Subst → cainele	Verb → doarme
	Subst → urs	
	Subst → ursul	
	Subst → pisica	
	Subst → pantera	

Rezolvare:

Atribute:

	Multime de valori	Obs.:
a	art, neart, any	a – articol : substantivul poate fi articulat (articol hotarat) sau nearticulat; any – forma comuna pentru articulat si nearticulat
gen	m, f, n	articolul nehotarat trebuie sa fie o sau un - corespunzator substantivului dat

Propozitie → NP VP .		NP – Noun Phrase VP – Verb Phrase
VP → Verb		
NP → Det Subst	Subst.a ≠ neart Subst.gen = Det.gen	Subst.a ≠ neart este o conditie; subst. poate fi nearticulat sau are forma comuna pentru forma articulata si nearticulata
NP → Subst		

Det → un	Det.gen←m	(Det – vine de la determiner, si il vom folosi pentru a “codifica” articolul nehotarat)
Det → o	Det.gen←f	
Subst → caine	Subst.a←neart Subst.gen ←m	

Subst → cainele	Subst.a ← art Subst.gen ← m	
Subst → urs	Subst.a ← neart Subst.gen ← m	
Subst → ursul	Subst.a ← art Subst.gen ← m	
Subst → pisica	Subst.a ← any Subst.gen ← f	
Subst → pantera	Subst.a ← any Subst.gen ← f	
Verb → alearga		
Verb → doarme		

9.

Dati o gramatica de attribute care traduce propozitii simple din limba romana in limba engleza, facand reordonarea substantive-adjectiv acolo unde este cazul.

Propozitiile in limba romana au structura: substantiv articulat + adjectiv + "sunt" + adjectiv

Gramatica va functiona pentru cuvintele: *merele, perele, rosii, verzi, sunt, delicioase*.

Exemplu:

Traducerea propozitiei: *merele rosii sunt delicioase* va fi *the red apples are delicious*.

Nu se cere verificarea ne-repetarii unui cuvant, de exemplu, gramatica poate functiona pentru propozitia: *merele rosii sunt rosii*.

Rezolvare:

(aici + - semnifica concatenare; " " sunt folosite pentru a ilustra faptul ca vom lucra cu siruri de caractere)

S → NP VP

S.en ← NP.en + " " + VP.en

NP → N ADJ

NP.noun ← N.en

NP.adj ← ADJ.en

NP.en ← "the " + NP.adj + " " + NP.noun

VP → V ADJ

VP.en ← V.en + " " + ADJ.en

N → merele

N.en ← "apples"

N → perele

N.en ← "pears"

ADJ → rosii

ADJ.en ← "red"

ADJ → verzi

ADJ.en ← "green"

ADJ → delicioase

ADJ.en ← "delicious"

V → sunt

V.en ← "are"

Cod intermediar

1. a) Fie secventa de instructiuni

A := B + C*D

B := B + C*D

D := B + C*D

- Traduceti in cod intermediar cu 3 adrese, reprezentare cvadrupe
b) codul poate fi optimizat? Traduceti in cod intermediar optimizat

Rezolvare:

- a) cod intermediar cu 3 adrese, reprezentare cvadrupe

operator	arg1	arg2	rez
...	...	...	...
*	C	D	T1
+	B	T1	T2
:=	T2		A
*	C	D	T3
-	B	T3	T4
:=	T4		B
*	C	D	T5
+	B	T5	T6
:=	T6		D

și apoi în cod intermediar cu 3 adrese reprezentare triplete

A := B + C*D

B := B + C*D

D := B + C*D

	operator	arg1	arg2
...	...	...	...
(90)	*	C	D
(91)	+	B	(90)
(92)	:=	A	(91)
(93)	*	C	D
(94)	+	B	(93)
(95)	:=	B	(94)
(96)	*	C	D
(97)	+	B	(96)
(98)	:=	D	(97)

- b)
Sugestii:
- C*D poate fi calculat o singura data si folosita valoarea in toate cele 3 instructiuni
 - Calculul pentru B+C*D din prima instructiune poate fi folosit si in a doua instructiune

2. a) Traduceti in cod intermediar :

a := 0

for i := 1 to 5 do begin

a := a+1;

i := i+1

end

- b) Care este valoarea lui a la iesirea din secventa de instructiuni?
Dar valoarea lui i?

- a) cod intermediar cu 3 adrese, reprezentare cvadrupe:

operator	arg1	arg2	rez
...	...	...	...
:=	0		a

et_for	:=	1		i
	g>	i	5	et_end
	+	a	1	T1
	:=	T1		a
	+	i	1	T2
	:=	T2		i
	+	i	1	i
et_end	goto			et_for
	...	...	...	...

Observatie:

- Exista variante de traduceri gandite astfel incat, daca i nu se modifica in interiorul ciclului for, la iesire i va avea ultima valoare de ciclare.